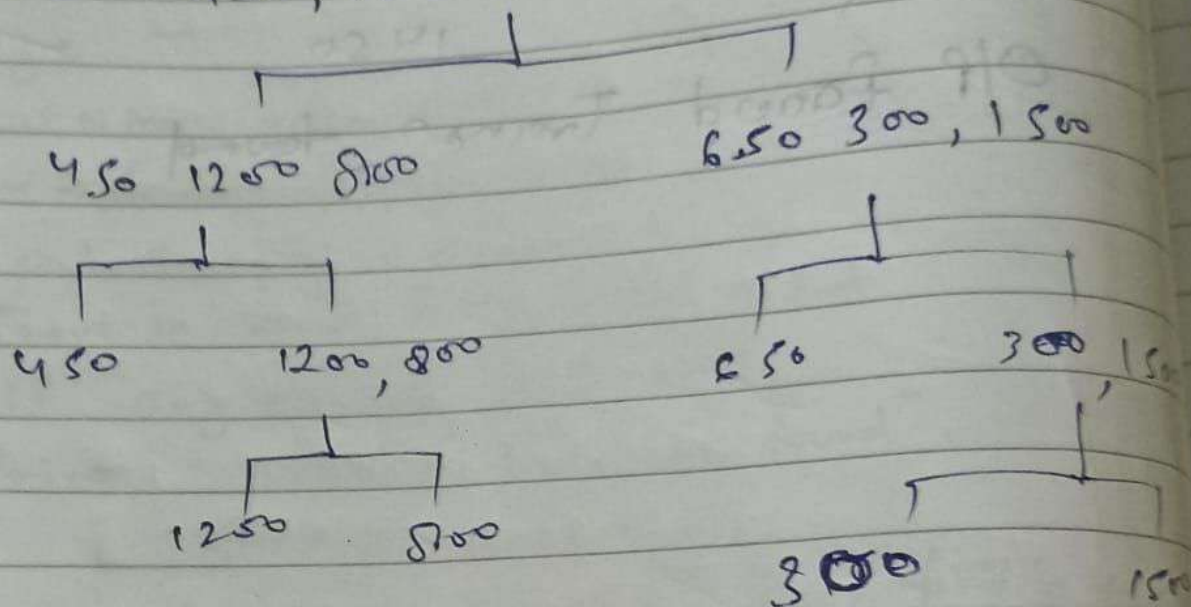


Q1 N package random
Sort → merge Sort.

Ex 450, 1200, 800, 650, 300, 1500

Q1-1
Divide



Q1-2 merge Sort

• merge

left	1200 > 800	450, 800, 1200	450, 800
Half	800 first	450 < 800	
	→ 800, 1200	450	

Right half	300, 1500	650, 300, 1500	300, 650, 1500
	300 < 1500	650 > 300	
	300, 1500	650 < 1500	

Left
450, 800, 1200

Right
300, 650, 1500

L	R	Smaller	O/P
450	300	300	300
450	650	450	300, 450
800	650	650	300, 450, 650
800	1500	800	300, 450, 650, 800
1200	1500	1200	300, 450, 650, 800, 1200
	1500	1500	[300, 450, 650, 800, 1200, 1500]

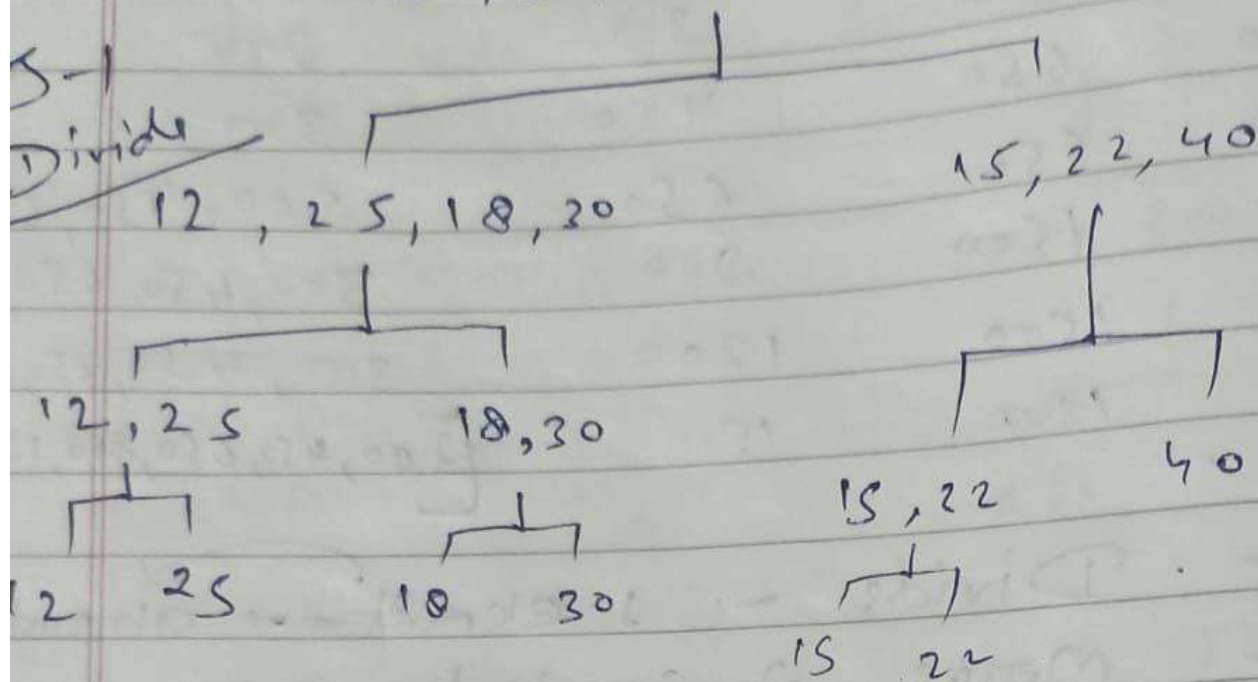
Divide → 1 element → already sorted
merge 2 sorted arr into one larger sorted arr.

Keep Merging until whole arr become sorted.

Merge Sort = Divide ↓↓↓
Then merge ↑↑↑

Q 2 merge Sort + Median + Count + Difference

Ex 12, 25, 18, 30, 15, 22, 40



3-2

merge

12, 25

12 < 18

25 > 18

12 < 30

25 < 30

18, 30

10, 30

12, 18, 25, 30

left

Right

15, 22, 40

15 < 40

22 < 40

15, 22, 40

12 < 15

18 > 15

25 > 22

30 < 40

12	15	18	22	25	30	40
0	1	2	3	4	5	6

median $\rightarrow$ 7 Element

$$n = 7 \rightarrow n / 2 = 3$$

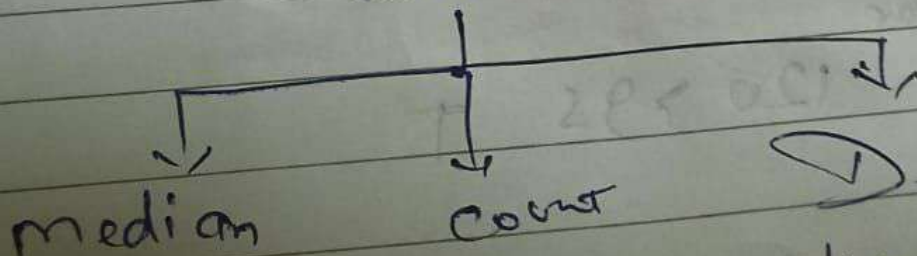
$$\text{Median} = \text{arr}[3] = 22$$

Count $\rightarrow$ element greater than 22.
25, 30, 40
count = 3

Difference Target - Smallest
 $40 - 12 = 28$

Approach $\rightarrow$ Take Arr :

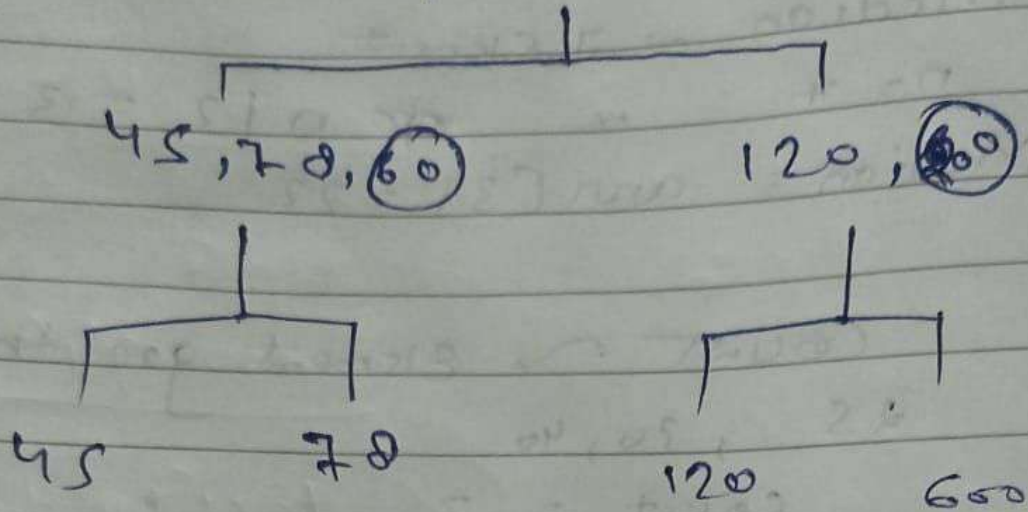
merge Sort
 $\rightarrow$ Sorted Arr



median (max - min)

Topic 2: Quick Sort

Q1 120, 45, 70, 200, 60, 95



Q-1



scan every element

keep track of posn

where next smaller element should go

$$i = -1 \quad | \quad j = 0$$

Q-2 Compare

$$i = -1$$

$$j = 0$$

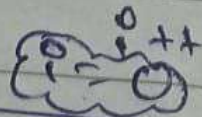
$$120 > 95 \quad \text{True}$$

$$i = -1$$

$$j = 1$$

$$45 < 95$$

Swap



arr[i] and arr[j]

45, 120, 70, 200, 60, 95

i

j

• $70 < 95$ $\swarrow$ swap
 $i++ \Rightarrow i = 1$

45, 70, 120, 200, 60, 95
 $\uparrow$ $\downarrow$

• 200
 $200 > 95$ Nothing

60
 $60 < 95$ swap
 $i = 2$

45, 70, 60, 200, 120, 95
 $\uparrow$

• Swap arr[i+1] with pivot
45, 70, 60, 95, 120, 200
 $\uparrow$
 pivot fix

45, 70, 60 < 95

95 < 120, 200

Quick Sort

Quick Sort

* left Recursion 45, 70, (60) pivot
partition $45 < 60$ ✓

Swap pivot 45, 60, 70
 $70 > 60$ ✗

classmate
Date _____
Page _____

* Right Recursion 120 200

$120 < 200 \quad \checkmark$

45, 60, 78, 95, 120, 200

Approach

Q5-1 Choose Pivot

Q5-2 Place pivot in its correct posn
by partition

S-3 Element smaller than pivot $\rightarrow$ left

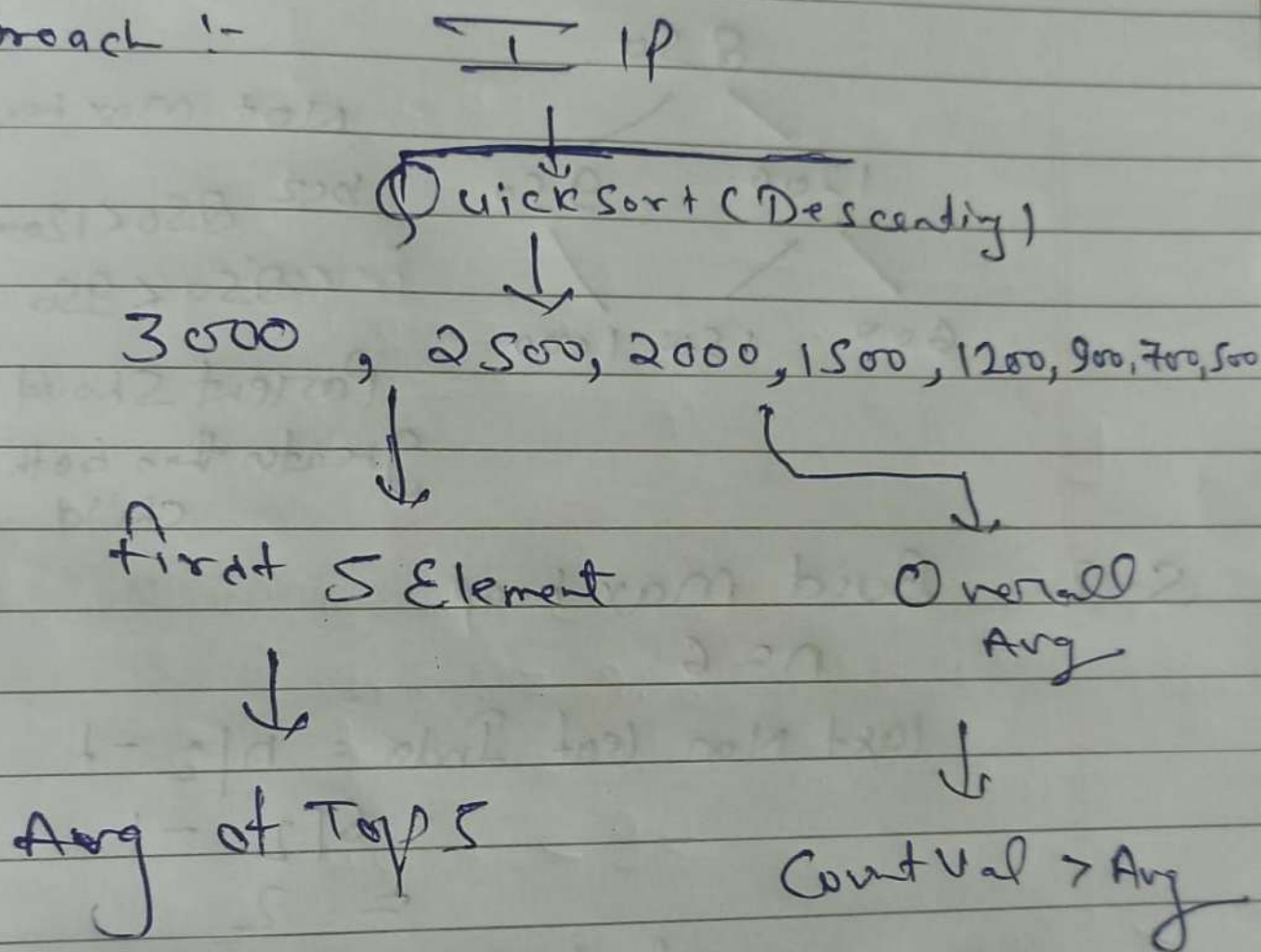
Q5-4 Element greater than pivot $\rightarrow$ right

S-5 Recursively repeat same process

Q2 Problem:

- ① Sort in desc Order $\rightarrow$ Quick Sort
- ② Print Top 5 highest val
- ③ find Avg of Top 5 val
- ④ count how many val are greater than overall avg.

Approach :-



Topic 3: Heap Sort

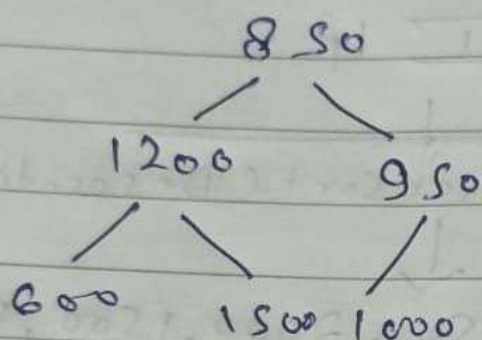
Q1

Ex 850, 1200, 950, 600, 1500, 1000
 0 1 2 3 4 5

Q1 - Convert Arr into Binary tree

Left child = $2 \times i + 1$

Right = $2 \times i + 2$



Not Max heap

bcz $850 < 1200$

$850 < 950$

Parent should
greater than both
child

Q2 - Build max heap
 $n = 6$

last Non leaf Index = $n/2 - 1$

= $6/2 - 1$

= 2

So heapify index

$2 + 1 \rightarrow 0$

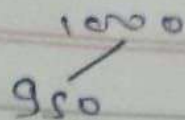
* Heapify Index 2

Node = ~~850~~ 950

Child: left = 1000

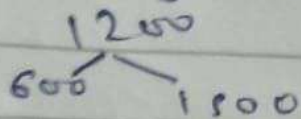
Right = None

1000 > 950 swap

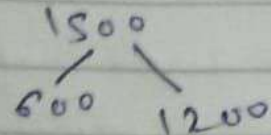


→ 850, 1200, 1000, 600, 1500, 950

* Heapify index = 1

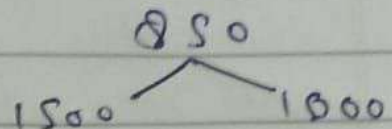
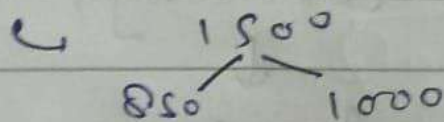


1200 < 1500

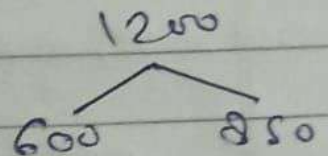
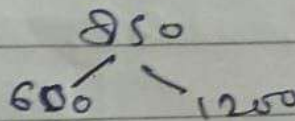


850, 1500, 1000, 600, 1200, 950

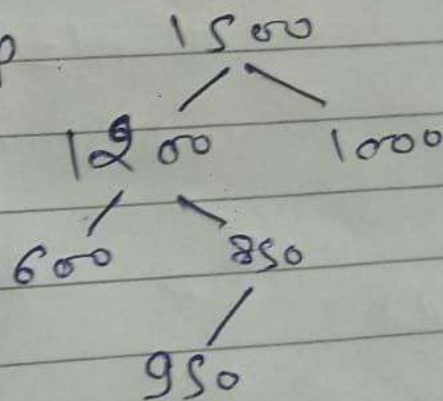
* Heapify Idx = 0



*



Max Heap



S-3

first Extn → swap root with

1500 ↔ 950

last element

950, 1200, 1000, 600, 850, 1500

Result

Q2

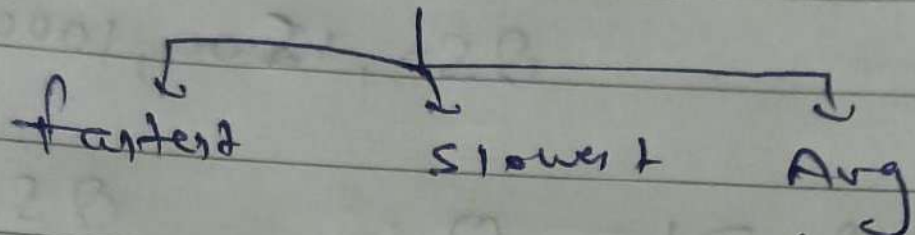
IP



Heap Sort



Sorted Arr



Count (< Avg)



Percentage